Deep Dive into Amazon Reviews: Sentiment Classification with Traditional ML and BERT

BY BLESSING OSUAGWU - GH1038684

Problem statement

Imagine Amazon with millions of customer reviews. It's impossible for people to read every single one to figure out what customers love or hate about products. But understanding these feelings, whether reviews are positive or negative, is super important for Amazon to make better products, help customers faster, and market things smarter.Given the immense volume of reviews, manual analysis is simply impractical, costly, and time-consuming, so the only way to tackle it is with an automated solution.

The Task Formulation and Process to solving it

This business challenge is precisely formulated as a binary text classification task within the domain of NLP. The central objective is to build an NLP system capable of automatically classifying a given Amazon product review text into one of two distinct sentiment categories: 'positive' or 'negative'.

To tackle this sentiment analysis challenge,we've sourced public data from Kaggle (https://www.kaggle.com/datasets/henryshan/amazon-review-sentiment-analysis),which contains(4915 entries and 12 columns, includes both numerical and textual data) then we'll design and implement an NLP pipeline following a standard machine learning workflow. This involved several key stages:

Data Loading and Preparation

Exploratory Data Analysis (EDA)

Preprocessing

Feature Engineering and Balancing

Model Training and Evaluation

Import essential Libraries and install some packages

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
import warnings

import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from wordcloud import WordCloud


from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE


!pip install -q transformers datasets
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification
from transformers import create_optimizer
import tensorflow as tf


warnings.filterwarnings('ignore')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.11/dist-packages (4.
3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.11/dist-
packages (from gensim) (1.26.4)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.11/dis
t-packages (from gensim) (1.13.1)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.11/dist-p
ackages (from gensim) (7.1.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.11/dist-packages (fro
m smart-open>=1.8.1->gensim) (1.17.2)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

Out[1]: True

Data Loading and Preparation

In [2]:
```
dt_f = pd.read_csv('/content/amazon_reviews.csv')
dt_f.head()
```

Out[2]:

	Unnamed: 0	reviewerName	overall	reviewText	reviewTime	day_diff	helpful_yes	helpf
0	0	NaN	4.0	No issues.	2014-07-23	138	0	
1	1	0mie	5.0	Purchased this for my device, it worked as adv...	2013-10-25	409	0	
2	2	1K3	4.0	it works as expected. I should have sprung for...	2012-12-23	715	0	
3	3	1m2	5.0	This think has worked out great.Had a diff. br...	2013-11-21	382	0	
4	4	2&1/2Men	5.0	Bought it with Retail Packaging, arrived legit...	2013-07-13	513	0	

In [3]:
```
dt_f = dt_f.rename(columns={
    'reviewText': 'the messages',
})
```

In [4]:
```
def map_sentiment(score):
    if score >= 4:
        return 'positive'
    elif score <= 2:
        return 'negative'
    else:
        return 'neutral'
dt_f['the notion'] = dt_f['overall'].apply(map_sentiment)
```

In [5]:
```
dt_f = dt_f[dt_f['the notion'] != 'neutral']
```

In [6]:
```
print("Missing values per column:")
dt_f.isnull().sum()
```

```
Missing values per column:
```

Out[6]:

	0
Unnamed: 0	0
reviewerName	1
overall	0
the messages	1
reviewTime	0
day_diff	0
helpful_yes	0
helpful_no	0
total_vote	0
score_pos_neg_diff	0
score_average_rating	0
wilson_lower_bound	0
the notion	0

dtype: int64

In [7]:
```
dt_f = dt_f.dropna(subset=['the messages'])
```

In [8]:
```
dt_f['reviewerName'] = dt_f['reviewerName'].fillna('Unknown')
```

In [9]:
```
print("\nDataset info after  being cleaned:")
dt_f.info()
```

```
Dataset info after  being cleaning:
<class 'pandas.core.frame.DataFrame'>
Index: 4772 entries, 0 to 4914
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Unnamed: 0            4772 non-null   int64
 1   reviewerName          4772 non-null   object
 2   overall               4772 non-null   float64
 3   the messages          4772 non-null   object
 4   reviewTime            4772 non-null   object
 5   day_diff              4772 non-null   int64
 6   helpful_yes           4772 non-null   int64
 7   helpful_no            4772 non-null   int64
 8   total_vote            4772 non-null   int64
 9   score_pos_neg_diff    4772 non-null   int64
 10  score_average_rating  4772 non-null   float64
 11  wilson_lower_bound    4772 non-null   float64
 12  the notion            4772 non-null   object
dtypes: float64(3), int64(6), object(4)
memory usage: 521.9+ KB
```

In [10]:
```
dt_f['label_numeric'] = dt_f['the notion'].map({'negative': 0, 'positive': 1})
```

In [11]:
```
print("\nDataset shape:", dt_f.shape)
print(dt_f[['the messages', 'the notion', 'label_numeric']].head())
```

```
Dataset shape: (4772, 14)
                                        the messages the notion  label_numeric
0                                        No issues.   positive              1
1  Purchased this for my device, it worked as adv...   positive              1
2  it works as expected. I should have sprung for...   positive              1
3  This think has worked out great.Had a diff. br...   positive              1
4  Bought it with Retail Packaging, arrived legit...   positive              1
```

Summary:

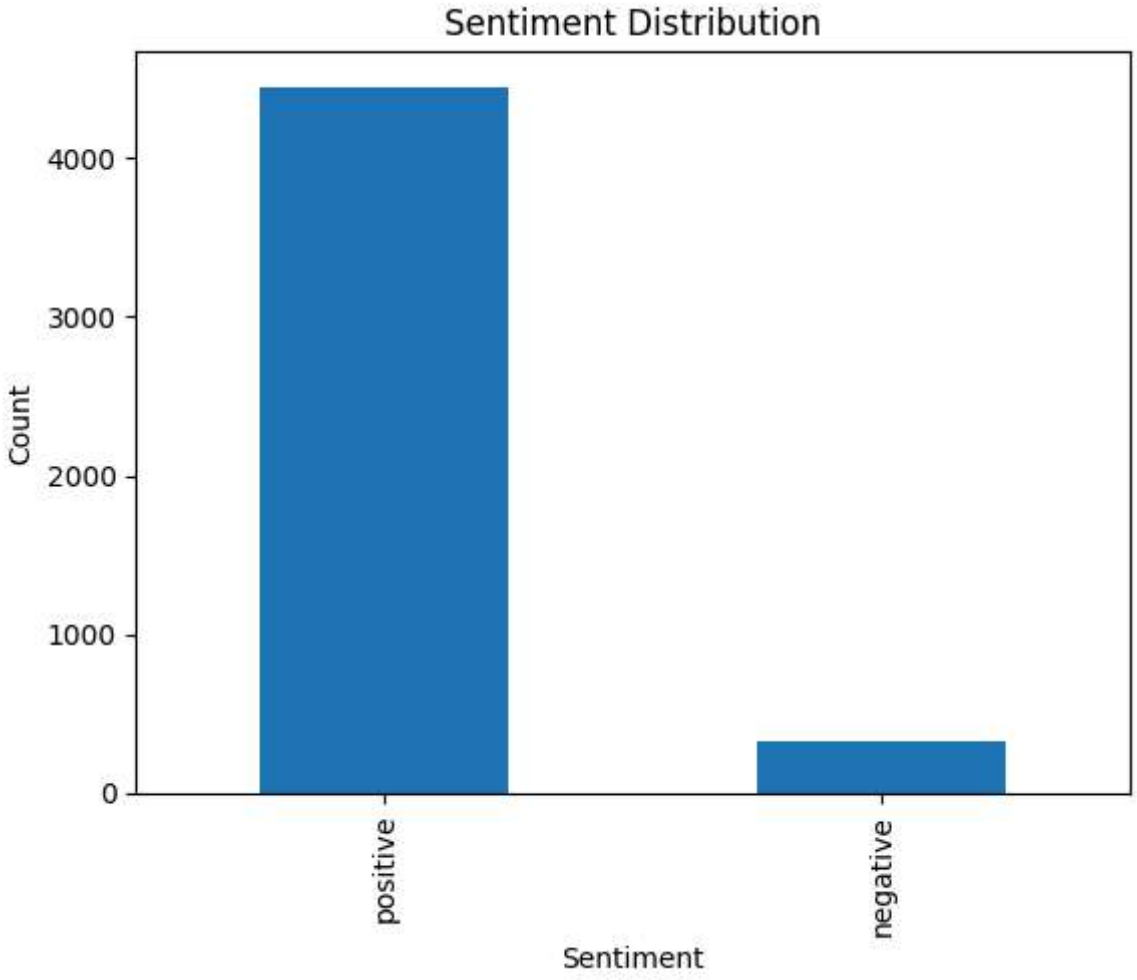
The dataset was loaded into a pandas DataFrame dt_f. The reviewText column was renamed to 'the messages'. A new column 'the notion' was created to categorize overall scores: scores ≥4 were mapped to 'positive', and scores ≤2 were mapped to 'negative', with 'neutral' sentiment being excluded. Missing values in the messages were dropped, and any missing reviewerName values were filled with 'Unknown'. Finally, a label_numeric column was added, mapping 'negative' to 0 and 'positive' to 1. After these cleaning steps, the dataset had a shape of 4772 rows and 14 columns.

Exploratory Data Analysis

In [12]:
```
print(dt_f['the notion'].value_counts())
```

```
the notion
positive    4448
negative     324
Name: count, dtype: int64
```

In [13]:
```
dt_f['the notion'].value_counts().plot(kind='bar', title='Sentiment Distribution')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.show()
```



In [14]:
```
print(f" Sentiment value count's percentage distribution:\n{round(dt_f['the notion'
```

```
 Sentiment value count's percentage distribution:
the notion
positive    93.21
negative     6.79
Name: count, dtype: float64
```

In [15]:
```
fig = plt.figure(figsize=(8,6))

colors = ["red", "green"]
explode = (0.1, 0.1)

plt.pie(
    dt_f['the notion'].value_counts(),
    labels=dt_f['the notion'].unique(),
```

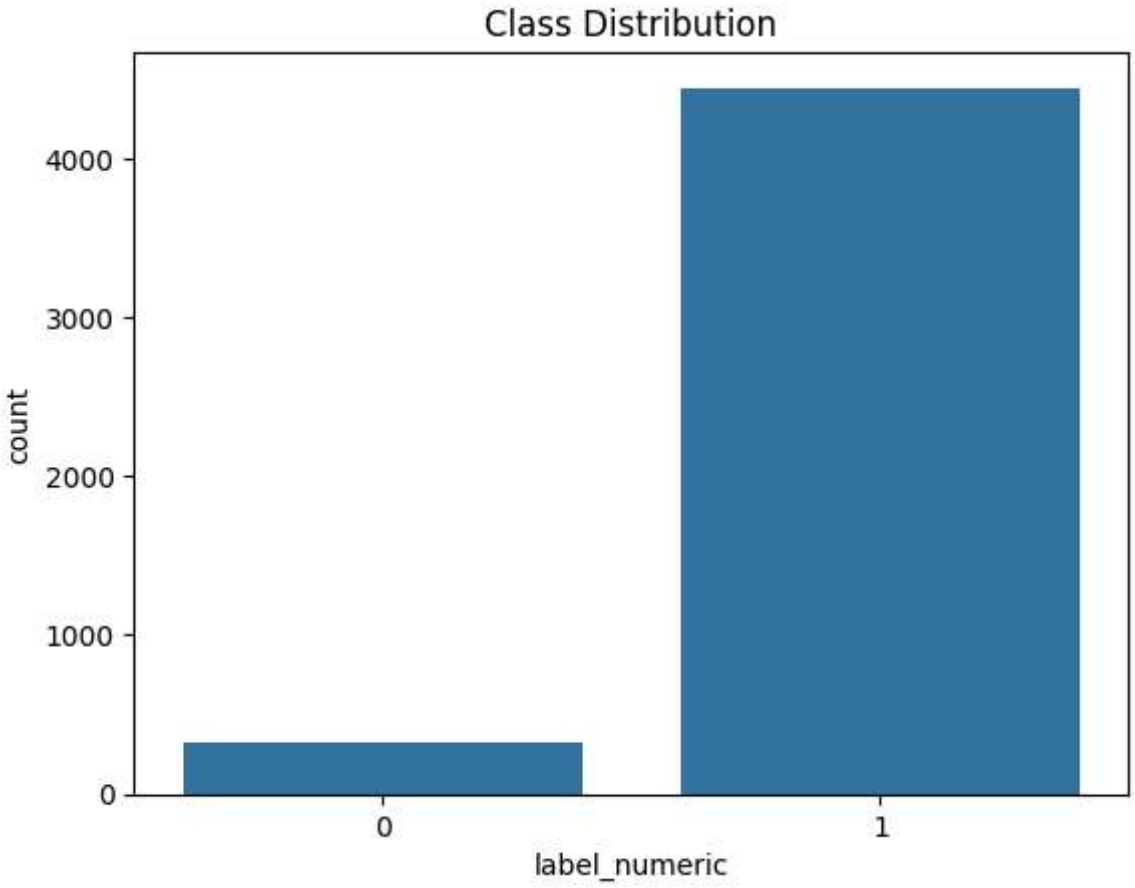
```
    colors=colors,
    explode=explode,
    autopct='%1.1f%%',
    shadow=True,
    startangle=140
)

plt.title("Sentiment Distribution")
plt.show()
```



In [16]:
```
dt_f['length'] = dt_f['the messages'].astype(str).apply(len)
```

In [17]:
```
sns.countplot(x='label_numeric', data=dt_f)
plt.title("Class Distribution")
plt.show()
```



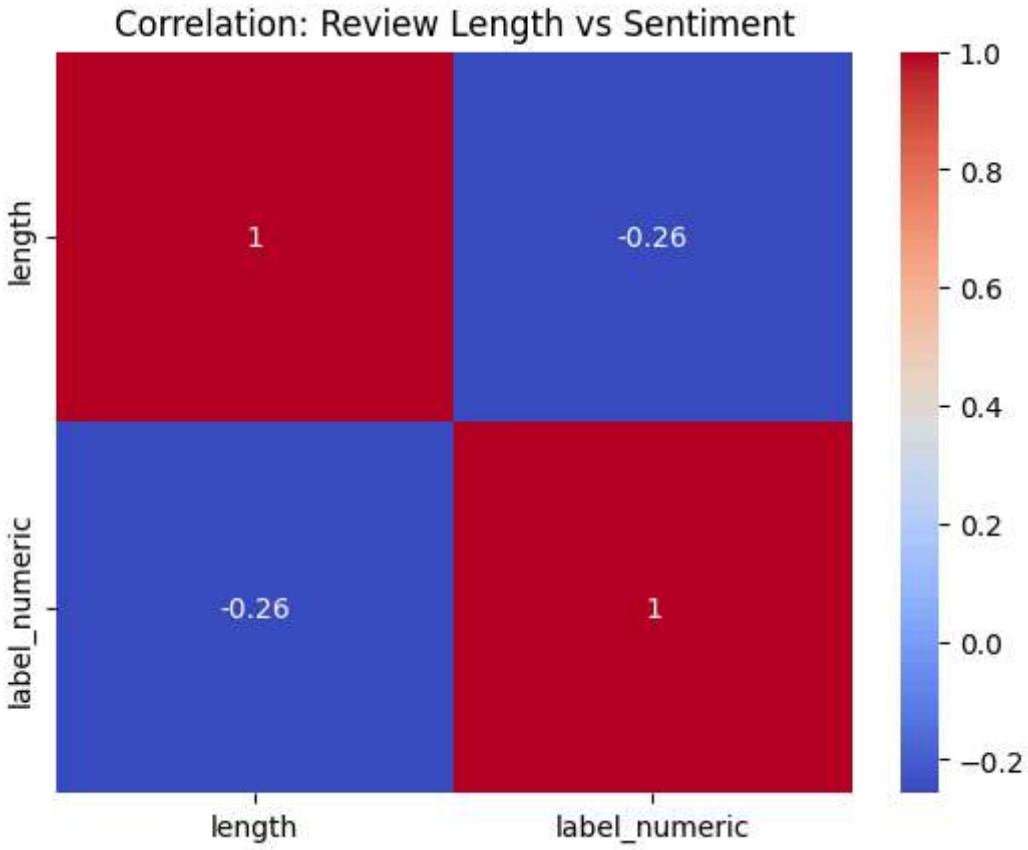
The focus on this section is to explore and understand the distribution of sentiments within the dataset. It reveals a significant class imbalance, with 4448 reviews classified as positive (93.21%) and only 324 as negative (6.79%). This imbalance is visually represented through both a bar plot and a pie chart. Additionally, a 'length' column was added to the dataset, which represents the character count of each review's message.

Regarding class distribution, a count plot further shows the imbalance in the 'label_numeric' column, where 0 represents negative sentiment and 1 represents positive sentiment.

Correlation Analysis

Are longer reviews more likely to be positive or negative?

In [18]:
```
corr = dt_f[['length', 'label_numeric']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation: Review Length vs Sentiment")
plt.show()
```

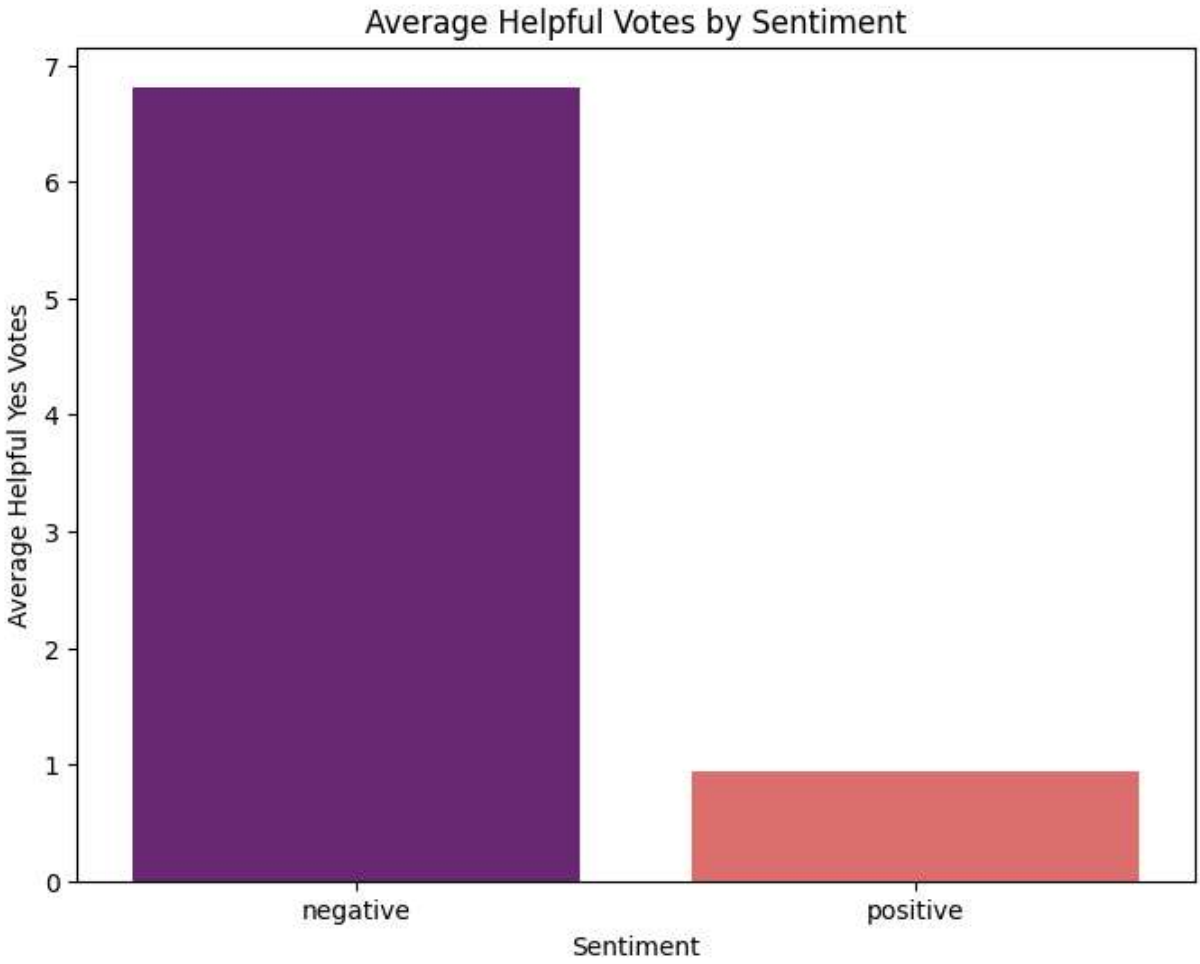


Longer reviews are slightly more likely to be negative, and shorter reviews are slightly more likely to be positive. However, since the correlation is close to zero (-0.26), review length is not a strong predictor of sentiment on its own.

Are positive reviews rated as more helpful, or are negative reviews more helpful to shoppers?

In [19]:
```
avg_helpful = dt_f.groupby('the notion')['helpful_yes'].mean().reset_index()

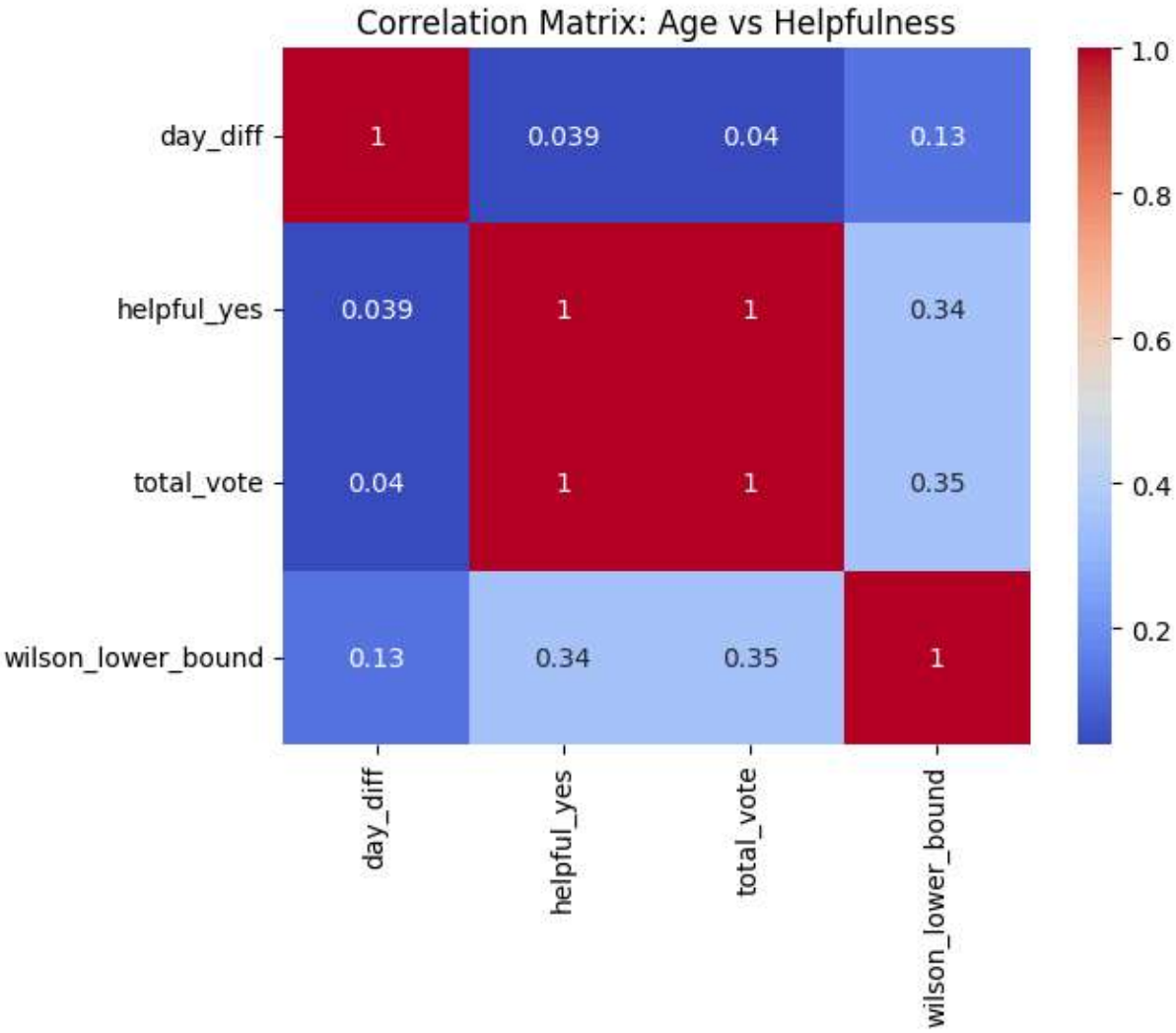
plt.figure(figsize=(8,6))
sns.barplot(data=avg_helpful, x='the notion', y='helpful_yes', palette='magma')
plt.title("Average Helpful Votes by Sentiment")
plt.ylabel("Average Helpful Yes Votes")
plt.xlabel("Sentiment")
plt.show()
```



The visualization indicates that negative reviews, on average, receive more helpful votes compared to positive reviews. This could imply that users find negative feedback or critical reviews more informative or helpful.

Do older reviews tend to have more helpful votes simply because they've been around longer?

In [20]:
```
corr = dt_f[['day_diff', 'helpful_yes', 'total_vote', 'wilson_lower_bound']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Matrix: Age vs Helpfulness")
plt.show()
```



There is a very weak positive correlation (0.039) between day_diff (age of review) and helpful_yes (number of helpful votes). This suggests that older reviews are only very slightly more likely to have helpful votes. Therefore, the data does not strongly support the idea that older reviews tend to have more helpful votes simply because they've been around longer. The relationship is negligible.

In [21]:
```
positive_text = " ".join(dt_f[dt_f['the notion'] == 'positive']['the messages'])

wordcloud_pos = WordCloud(width=400, height=300, background_color='white').generate
plt.figure(figsize=(8,7))
plt.imshow(wordcloud_pos, interpolation='bilinear')
plt.axis('off')
plt.title("Positive Reviews")
plt.show()
```



In [22]:
```
negative_text = " ".join(dt_f[dt_f['the notion'] == 'negative']['the messages'])

wordcloud_neg = WordCloud(width=400, height=300, background_color='white').generate
plt.figure(figsize=(8,7))
plt.imshow(wordcloud_neg, interpolation='bilinear')
plt.axis('off')
plt.title("Negative Reviews")
plt.show()
```



Made a visualization with WordCloud to see some of the positive and Negative reviews of the most frequent words in each sentiment category.

Preprocessing

In [23]:
```
stop_words = set(stopwords.words('english'))
Pts = PorterStemmer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+|https\S+", '', text)
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [Pts.stem(word) for word in tokens if word not in stop_words]
    return ' '.join(tokens)

dt_f['cleaned'] = dt_f['the messages'].astype(str).apply(clean_text)
print(dt_f[['the messages', 'cleaned']].head())
```

```
                                          the messages  \
0                                           No issues.
1  Purchased this for my device, it worked as adv...
2  it works as expected. I should have sprung for...
3  This think has worked out great.Had a diff. br...
4  Bought it with Retail Packaging, arrived legit...

                                              cleaned
0                                                issu
1  purchas devic work advertis never much phone m...
2  work expect sprung higher capac think made bit...
3  think work greathad diff bran gb card went sou...
4  bought retail packag arriv legit orang envelop...
```

Feature Engineering and Balancing

In [24]:
```
X = dt_f['cleaned']
y = dt_f['label_numeric']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
print("\nTraining set sentiment distribution:\n", y_train.value_counts())
print("\nTesting set sentiment distribution:\n", y_test.value_counts())
```

```
Training set shape: (3817,)
Testing set shape: (955,)

Training set sentiment distribution:
 label_numeric
1    3558
0     259
Name: count, dtype: int64

Testing set sentiment distribution:
 label_numeric
1    890
0     65
Name: count, dtype: int64
```

In [25]:
```
tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

In [28]:
```
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_tfidf, y_train)

print("Training set shape after SMOTE:", X_train_smote.shape)
print("\nTraining set sentiment distribution after SMOTE:\n", pd.Series(y_train_smo
```

```
Training set shape after SMOTE: (7116, 6386)

Training set sentiment distribution after SMOTE:
 label_numeric
1    3558
0    3558
Name: count, dtype: int64
```

Summary:

Cleaned the data, splitted the data to test and train, used SMOTE to handle class imbalance and transformed text into numerical TF-IDF features.

Model Training and Evaluation

Logistic Regression, Support Vector Machine, Naive Bayes

In [29]:
```
lr_model = LogisticRegression(solver='liblinear', random_state=42)
lr_model.fit(X_train_smote, y_train_smote)
```

Out[29]:
```
LogisticRegression
LogisticRegression(random_state=42, solver='liblinear')
```

In [30]:
```
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train_smote, y_train_smote)
```

Out[30]:
```
SVC
SVC(kernel='linear', random_state=42)
```

In [31]:
```
nb_model = MultinomialNB()
nb_model.fit(X_train_smote, y_train_smote)
```

Out[31]:
```
MultinomialNB
MultinomialNB()
```

In [32]:
```
lr_pred = lr_model.predict(X_test_tfidf)
print("Logistic Regression:")
print(classification_report(y_test, lr_pred))
```

```
Logistic Regression:
              precision    recall  f1-score   support

           0       0.66      0.75      0.71        65
           1       0.98      0.97      0.98       890

    accuracy                           0.96       955
   macro avg       0.82      0.86      0.84       955
weighted avg       0.96      0.96      0.96       955
```

In [33]:
```
svm_pred = svm_model.predict(X_test_tfidf)
print("Support Vector Machine:")
print(classification_report(y_test, svm_pred))
```

```
Support Vector Machine:
              precision    recall  f1-score   support

           0       0.77      0.68      0.72        65
           1       0.98      0.99      0.98       890

    accuracy                           0.96       955
   macro avg       0.87      0.83      0.85       955
weighted avg       0.96      0.96      0.96       955
```

In [34]:
```
nb_pred = nb_model.predict(X_test_tfidf)
print("Naive Bayes:")
print(classification_report(y_test, nb_pred))
```

```
Naive Bayes:
              precision    recall  f1-score   support

           0       0.42      0.88      0.56        65
           1       0.99      0.91      0.95       890

    accuracy                           0.91       955
   macro avg       0.70      0.89      0.76       955
weighted avg       0.95      0.91      0.92       955
```

BERT

In [35]:
```
df_bert = dt_f[['the messages', 'label_numeric']]
train_texts, test_texts, train_labels, test_labels = train_test_split(
    df_bert['the messages'],
    df_bert['label_numeric'],
    test_size=0.2,
    stratify=df_bert['label_numeric'],
    random_state=42
)

print(f"Train texts shape: {train_texts.shape}, Test texts shape: {test_texts.shape
print(f"Train labels shape: {train_labels.shape}, Test labels shape: {test_labels.s
```

```
Train texts shape: (3817,), Test texts shape: (955,)
Train labels shape: (3817,), Test labels shape: (955,)
```

In [39]:
```
BATCH_SIZE = 32
MAX_LENGTH = 128
MODEL_NAME = 'bert-base-uncased'
EPOCHS = 3
LEARNING_RATE = 2e-5

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = TFAutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels
```

```
All PyTorch model weights were used when initializing TFBertForSequenceClassificatio
n.

Some weights or buffers of the TF 2.0 model TFBertForSequenceClassification were not
initialized from the PyTorch model and are newly initialized: ['classifier.weight',
'classifier.bias']
You should probably TRAIN this model on a down-stream task to be able to use it for
predictions and inference.
```

In [40]:
```
train_encodings = tokenizer(list(train_texts), truncation=True, padding=True, max_l
test_encodings = tokenizer(list(test_texts), truncation=True, padding=True, max_len

train_labels_one_hot = tf.one_hot(train_labels, depth=2)
test_labels_one_hot = tf.one_hot(test_labels, depth=2)
```

In [41]:
```
train_dataset = tf.data.Dataset.from_tensor_slices((dict(train_encodings), train_la
test_dataset = tf.data.Dataset.from_tensor_slices((dict(test_encodings), test_label

num_train_steps = len(train_dataset) * EPOCHS
optimizer, schedule = create_optimizer(init_lr=LEARNING_RATE, num_warmup_steps=0, n

model.compile(optimizer=optimizer,
              loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

In [42]:
```
model.fit(train_dataset,
          epochs=EPOCHS,
          validation_data=test_dataset)
```

```
Epoch 1/3
120/120 [==============================] - 6064s 50s/step - loss: 0.2025 - accuracy:
0.9303 - val_loss: 0.0994 - val_accuracy: 0.9571
Epoch 2/3
120/120 [==============================] - 5959s 50s/step - loss: 0.0705 - accuracy:
0.9741 - val_loss: 0.0707 - val_accuracy: 0.9759
Epoch 3/3
120/120 [==============================] - 5907s 49s/step - loss: 0.0279 - accuracy:
0.9914 - val_loss: 0.0783 - val_accuracy: 0.9770
```

Out[42]:
```
<tf_keras.src.callbacks.History at 0x7c75c8bbd0d0>
```

In [43]:
```
bert_pred_probs = model.predict(test_dataset).logits
bert_pred = tf.argmax(bert_pred_probs, axis=1).numpy()

print("BERT Model:")
print(classification_report(test_labels, bert_pred))
```

```
print("Confusion Matrix:")
print(confusion_matrix(test_labels, bert_pred))
```

```
30/30 [==============================] - 472s 16s/step
BERT Model:
              precision    recall  f1-score   support

           0       0.86      0.78      0.82        65
           1       0.98      0.99      0.99       890

    accuracy                           0.98       955
   macro avg       0.92      0.89      0.91       955
weighted avg       0.98      0.98      0.98       955

Confusion Matrix:
[[ 51  14]
 [  8 882]]
```

Summary of the performance of each model on the test set:

Model	Accuracy	Precision (Negative)	Recall (Negative)	F1-Score (Negative)	Precision (Positive)	Recall (Positive)	F1-Score (Positive)
Logistic Regression	0.96	0.66	0.75	0.71	0.98	0.97	0.98
Support Vector Machine	0.96	0.77	0.68	0.72	0.98	0.99	0.98
Naive Bayes	0.91	0.42	0.88	0.56	0.99	0.91	0.95
BERT	0.98	0.86	0.78	0.82	0.98	0.99	0.99

Final Summary

This (NLP) pipeline designed to perform sentiment analysis on Amazon customer reviews. The core objective is to automatically classify reviews as either 'positive' or 'negative', providing valuable insights for business improvement.

The pipeline begins with Data Loading and Preparation, where the Amazon reviews dataset is imported. Raw review texts are standardized, numerical ratings are converted into clear sentiment categories ('positive' or 'negative' with neutral reviews excluded), and missing data is addressed. This results in a clean dataset of 4772 reviews ready for analysis.

Exploratory Data Analysis (EDA) highlights a significant challenge: a high class imbalance, with positive reviews outnumbering negative ones by a ratio of approximately 14:1 (93.21% positive vs. 6.79% negative). Additional analysis involves creating a 'length' feature for each review and performing Correlation Analysis. This reveals a weak negative correlation

between review length and sentiment, indicating that longer reviews are only slightly more likely to be negative. Intriguingly, negative reviews tend to receive more 'helpful' votes on average, although the age of a review shows a negligible correlation with its helpfulness. Word clouds are also generated to visually summarize frequently used terms in positive and negative reviews.

The Preprocessing stage cleans the review text by converting it to lowercase, removing URLs and non-alphabetic characters, tokenizing words, removing common stopwords, and applying stemming. This cleaned text is then used for Data Splitting and Vectorization, where the dataset is divided into training and testing sets, and the text is transformed into numerical TF-IDF features. To address the severe class imbalance, SMOTE (Synthetic Minority Over-sampling Technique) is applied to the training data, effectively balancing the number of positive and negative samples for robust model training.

Finally, the Model Training and Evaluation phase explores both traditional machine learning models and a deep learning approach:

Traditional Models: Logistic Regression, Support Vector Machine, and Multinomial Naive Bayes are trained on the balanced TF-IDF features and evaluated on the imbalanced test set. All these models achieve high overall accuracy due to the large positive class, but their performance on the minority negative class varies.

Support Vector Machine shows the highest precision for negative reviews (0.77), indicating fewer false positives.

Naive Bayes achieves the highest recall for negative reviews (0.88), meaning it identifies a larger proportion of actual negative reviews.

Logistic Regression offers a good balance between precision and recall for the negative class.

BERT Model: A pre-trained bert-base-uncased model is fine-tuned for sequence classification. This neural network demonstrates superior overall performance. BERT achieves the highest F1-score (0.82) for the minority (negative) class, while maintaining excellent performance on the majority (positive) class.

In conclusion, while traditional models provide reasonable sentiment classification, the BERT model emerges as the most effective solution for this task, excelling in accurately identifying negative reviews despite the initial dataset imbalance. This NLP pipeline provides a strong foundation for automated customer feedback analysis, offering actionable insights for Amazon's business operations.

REFERENCES

https://www.kaggle.com/datasets/henryshan/amazon-review-sentiment-analysis

Kumar, L.K. et al. (2024) 'Analyzing public sentiment on the Amazon website: A GSK-Based Double Path Transformer Network Approach for Sentiment analysis,' IEEE Access, 12, pp. 28972–28987. https://doi.org/10.1109/access.2024.3368441.

Ali, H. et al. (2024) 'Analyzing Amazon Products Sentiment: A comparative study of machine and deep learning, and Transformer-Based techniques,' Electronics, 13(7), p. 1305. https://doi.org/10.3390/electronics13071305.

Mao, Y., Liu, Q. and Zhang, Y. (2024) 'Sentiment analysis methods, applications, and challenges: A systematic literature review,' Journal of King Saud University - Computer and Information Sciences, 36(4), p. 102048. https://doi.org/10.1016/j.jksuci.2024.102048.

Jim, J.R. et al. (2024) 'Recent advancements and challenges of NLP-based sentiment analysis: A state-of-the-art review,' Natural Language Processing Journal, 6, p. 100059. https://doi.org/10.1016/j.nlp.2024.100059.

Wu, Y. et al. (2024) 'Research on the application of deep learning-based BERT model in sentiment analysis,' arXiv (Cornell University) [Preprint]. https://doi.org/10.48550/arxiv.2403.08217.